

机器人强化学习的平滑探索

Antonin Raffin

Robotics and Mechatronics Center (RMC)
German Aerospace Center (DLR) Germany
antonin.raffin@dlr.de

Jens Kober

Cognitive Robotics Department
Delft University of Technology The Netherlands
j.kober@tudelft.nl

Frek Stulp

Robotics and Mechatronics Center (RMC)
German Aerospace Center (DLR) Germany
frek.stulp@dlr.de

摘要:

强化学习 (Reinforcement Learning, RL) 使机器人能够通过真实世界的交互来学习技能。在实践中, 深度强化学习 (Deep RL) 所使用的非结构化逐步探索——在仿真中往往非常成功——会导致真实机器人产生不平稳的运动模式。由此产生的抖动行为会带来探索效果差, 甚至损坏机器人的后果。我们通过将状态依赖探索 (State-Dependent Exploration, SDE) [1] 适配到当前的深度强化学习算法来解决这些问题。为了实现这一适配, 我们对原始 SDE 提出了两项扩展: 使用更通用的特征以及周期性重采样噪声, 从而得到一种新的探索方法——广义状态依赖探索 (generalized State-Dependent Exploration, gSDE)。我们在仿真中 (PyBullet 连续控制任务) 以及直接在三种不同的真实机器人上 (肌腱驱动机器人、四足机器人和遥控车) 对 gSDE 进行了评估。gSDE 的噪声采样间隔允许在性能和动作平滑性之间取得折中, 从而能够直接在真实机器人上进行训练而不损失性能。代码可在 <https://github.com/DLR-RM/stable-baselines3> 获取。

1 引言

最早使用人工智能方法的机器人之一被称为“沙基” (Shakey), 因为它在运行时会剧烈摇晃 [2]。如今, 摇晃在机器人学中再次变得相当普遍, 但原因不同。在使用深度强化学习 (DeepRL) 学习机器人技能时, 探索的事实标准是在每个时间步独立地从高斯分布中采样一个噪声向量 ϵ_t , 然后将其添加到策略输出中。这种方法产生的噪声类型如图 1 左侧所示, 在仿真中可能非常有效 [3, 4, 5, 6, 7]。

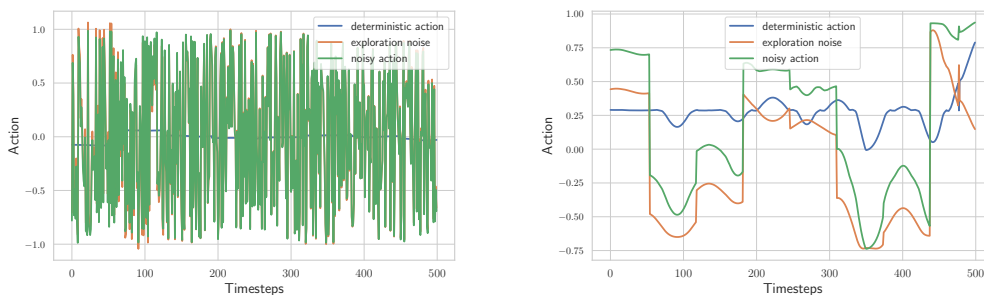


图 1: 左: 非结构化探索, 通常用于仿真强化学习。右: gSDE 提供平滑且一致的探索。

非结构化探索也已应用于机器人技术 [8, 9]。但在真实机器人上进行实验时，它存在许多已被反复指出的缺点 [1, 10, 11, 12, 13]：1) 每一步独立采样会导致行为抖动 [14]，轨迹嘈杂且不稳定。2) 急促的运动模式可能损坏真实机器人的电机，并增加磨损。3) 在现实世界中，系统充当低通滤波器。因此，连续的扰动可能相互抵消，导致探索效果不佳。在高控制频率下尤其如此 [15]。4) 它会导致较大的方差，且方差随时间步数增加而增大 [10, 11, 12]。实际上，我们在三台真实机器人上观察到了所有这些缺点，包括图 4a 所示的肌腱驱动机器人大卫，它是本工作使用的主要实验平台。从所有实用角度来看，带有非结构化噪声的深度强化学习无法应用于大卫。

在机器人学中，已提出多种解决方案来应对非结构化噪声的低效性。这些方案包括相关噪声 [8, 15]、低通滤波器 [16, 17]、动作重复 [18] 或底层控制器 [16, 9]。一种更具原则性的解决方案是在参数空间而非动作空间中进行探索 [19, 20]。这种方法通常需要对算法进行根本性修改，并且在参数数量较多时更难调节。

状态依赖探索 (SDE) [1, 11] 被提出作为在参数空间和动作空间探索之间的折中方案。SDE 将采样噪声替换为状态依赖的探索函数，该函数在一个回合内对给定状态返回相同的动作。这导致探索更平滑，每个回合的方差更小。

据我们所知，尚无深度强化学习算法成功与 SDE 结合。我们推测这是因为它所解决的问题——抖动、急促的运动——在仿真中并不明显，而仿真目前是社区关注的焦点。

在本文中，我们旨在重新激发对 SDE 的兴趣，将其作为一种有效方法，用于解决在真实机器人上使用独立采样高斯噪声所产生的探索问题。我们的具体贡献也决定了本文的结构，如下：

1. 强调非结构化高斯探索存在的问题（第 1 节）。2. 将随机微分方程 (SDE) 适配到近期的深度强化学习算法中，并解决原始公式中的一些问题（第 2.2 节和第 3 节）。3. 评估不同方法在平滑性与性能之间的折衷，并展示噪声采样间隔的影响（第 4.1 节和第 4.2 节）。4. 成功地将强化学习直接应用于三个真实机器人：肌腱驱动机器人、四足机器人和遥控车，无需模拟器或滤波器（第 4.3 节）。

2 背景

在强化学习中，智能体与其环境交互，通常建模为马尔可夫决策过程 (MDP) $(\mathcal{S}, \mathcal{A}, p, r)$ ，其中 $\mathcal{S}$ 是状态空间， $\mathcal{A}$ 是动作空间， $p(s'|s, a)$ 是转移函数。在每一步 t ，智能体遵循其策略 $\pi: \mathcal{S} \mapsto \mathcal{A}$ 在状态 s 下执行动作 a 。然后在下一个状态 s' 中收到反馈信号：奖励 $r(s, a)$ 。智能体的目标是最大化长期奖励。更正式地说，目标是最大化折扣奖励之和的期望，在由其策略 π 生成的轨迹 ρ_π 上：
$$\mathbb{E}_{(s_t, a_t) \sim \rho_\pi} \left[\sum_t \gamma^t r(s_t, a_t) \right] \quad (1)$$

其中 $\gamma \in [0, 1)$ 是折扣因子，表示在最大化短期奖励和长期奖励之间的权衡。智能体与环境的交互通常被分解为称为回合的序列，当智能体到达终止状态时结束。

2.1 动作空间或策略参数空间中的探索

在连续动作的情况下，探索通常在动作空间 [21, 22, 23, 24, 25, 5] 中进行。在每个时间步，噪声向量 ϵ_t 独立地从高斯分布中采样，然后添加到控制器输出：

$$a_t = \mu(s_t; \theta_\mu) + \epsilon_t, \quad \epsilon_t \sim \mathcal{N}(0, \sigma^2) \quad (2)$$

其中 $\mu(\mathbf{s}_t)$ 是确定性策略， $\pi(\mathbf{a}_t|\mathbf{s}_t) \sim \mathcal{N}(\mu(\mathbf{s}_t), \sigma^2)$ 是用于探索的随机策略。 θ_μ 表示确定性策略的参数。为简单起见，在本文中，我们只考虑具有对角协方差矩阵的高斯分布。因此，这里 σ 是一个与动作空间 $\mathcal{A}$ 维度相同的向量。

或者，探索也可以在参数空间 [11, 19, 20] 中进行：

$$\mathbf{a}_t = \mu(\mathbf{s}_t; \theta_\mu + \epsilon), \quad \epsilon \sim \mathcal{N}(0, \sigma^2) \quad (3)$$

在回合开始时，扰动 ϵ 被采样并加到策略参数上。这通常带来更一致的探索，但随着参数数量 θ_μ 增加而变得困难 [19]。

2.2 状态依赖探索

状态依赖探索（State-Dependent Exploration, SDE）[1, 11] 是一种中间方案，它将噪声作为状态的函数 $\mathbf{s}_t$ 加到确定性动作 $\mu(\mathbf{s}_t)$ 上。在回合开始时，该探索函数的参数 θ_ϵ 从高斯分布中抽取。得到的动作 $\mathbf{a}_t$ 如下：

$$\mathbf{a}_t = \mu(\mathbf{s}_t; \theta_\mu) + \epsilon(\mathbf{s}_t; \theta_\epsilon), \quad \theta_\epsilon \sim \mathcal{N}(0, \sigma^2) \quad (4)$$

这种基于回合的探索比无结构的逐步探索更平滑、更一致。因此，在一个回合内，对于给定状态 $\mathbf{s}$ ，动作 $\mathbf{a}$ 不会围绕均值振荡，而是保持不变。

SDE 不应与无结构噪声混淆，后者的方差可以依赖状态，但噪声仍在每一步采样，如 SAC 中的情况。

在本文其余部分，为避免符号过载，我们省略时间下标 t , i.e. we，现在用 $\mathbf{s}$ 代替 $\mathbf{s}_t$ 。 $\mathbf{s}_j$ 或 $\mathbf{a}_j$ 来表示状态或动作向量的元素。在线性探索函数 $\epsilon(\mathbf{s}; \theta_\epsilon) = \theta_\epsilon \mathbf{s}$ 的情况下，通过对高斯分布的运算，Rückstieß 等人 [1] 证明动作元素 $\mathbf{a}_j$ 服从正态分布：

$$\pi_j(\mathbf{a}_j|\mathbf{s}) \sim \mathcal{N}(\mu_j(\mathbf{s}), \hat{\sigma}_j^2) \quad (5)$$

其中 $\hat{\sigma}$ 是对角矩阵，其元素为 $\hat{\sigma}_j = \sqrt{\sum_i (\sigma_{ij} \mathbf{s}_i)^2}$ 。知道策略分布，可以得到对数似然 $\log \pi(\mathbf{a}|\mathbf{s})$ 关于方差 σ 的导数：

$$\frac{\partial \log \pi(\mathbf{a}|\mathbf{s})}{\partial \sigma_{ij}} = \frac{(\mathbf{a}_j - \mu_j)^2 - \hat{\sigma}_j^2}{\hat{\sigma}_j^3} \frac{\mathbf{s}_i^2 \sigma_{ij}}{\hat{\sigma}_j} \quad (6)$$

这可以很容易地代入似然比梯度估计器 [26]，从而在训练过程中自适应 σ 。因此，SDE 与标准策略梯度方法兼容，同时解决了无结构探索的大部分缺点。

对于非线性探索函数，所得分布 $\pi(\mathbf{a}|\mathbf{s})$ 在多数情况下是未知的。因此，计算关于方差的精确导数并非易事，可能需要近似推断。鉴于我们注重简洁性，将此扩展留待未来工作。

3 广义状态依赖探索

考虑式 (5) 和式 (7)，原始公式的若干局限性显而易见：

- i 噪声在一个回合内不发生变化，若回合长度较长则存在问题，因为探索将受限 [27]。
- ii 策略的方差 $\hat{\sigma}_j = \sqrt{\sum_i (\sigma_{ij} \mathbf{s}_i)^2}$ 依赖于状态空间维度（随维度增长），这意味着初始 σ 必须针对每个问题进行调优。
- iii 状态与探索噪声之间仅存在线性依赖，限制了可能性。

iv 状态必须归一化，因为梯度和噪声幅度依赖于状态幅度。

为缓解上述问题并使其适应深度强化学习算法，本文提出两项改进：

1. 我们每 n 步采样探索函数的参数 θ_ϵ ，而非每个回合。
2. 实际上，我们可以使用任意特征代替状态 $\mathbf{s}$ 。我们选择策略特征 $\mathbf{z}_\mu(\mathbf{s}; \theta_{\mathbf{z}_\mu})$ （确定性输出 $\mu(\mathbf{s}) = \theta_\mu \mathbf{z}_\mu(\mathbf{s}; \theta_{\mathbf{z}_\mu})$ 前的最后一层）作为噪声函数 $\epsilon(\mathbf{s}; \theta_\epsilon) = \theta_\epsilon \mathbf{z}_\mu(\mathbf{s})$ 的输入。

每 n 步采样参数 θ_ϵ 解决了问题 i，并产生一个统一框架 [27]，该框架同时涵盖非结构化探索（ $n=1$ ）和原始随机微分方程（ n =回合长度）。尽管此公式遵循深度强化学习算法每 m 步更新参数的描述，但这一关键参数对平滑性和性能的影响至今被忽视。

利用策略特征可以缓解问题二、三和四：状态 $\mathbf{s}$ 与噪声 ϵ 之间的关系是非线性的，策略的方差仅取决于网络架构，例如允许使用图像作为输入。因此，这种表述更为通用，并且当使用状态作为噪声函数的输入或策略为线性时，包含了原始的随机微分方程描述。

我们将由此得到的方法称为广义状态依赖探索（gSDE）。

深度强化学习算法 将这一更新版的随机微分方程集成到近期的深度强化学习算法中（如附录所列）是直接的。对于依赖概率分布的算法，如软演员-评论家（SAC）或近端策略优化（PPO），我们可以用公式（5）中的分布替换原始的高斯分布，其中对数似然的解析形式已知（参见公式（7））。

4 实验

在本节中，我们研究广义状态依赖探索以回答以下问题：

- 广义状态依赖探索与原始随机微分方程相比如何？每项提出的修改有何影响？
- 在平滑性与性能之间的权衡方面，广义状态依赖探索与其他类型的探索噪声相比如何？
- 广义状态依赖探索在真实系统上的表现如何？

4.1 平滑性与性能之间的折衷

实验设置 为了在性能与平滑性的折衷方面将 gSDE 与其他类型的探索进行比较，我们从 PyBullet[28] 环境中选择了 4 个运动任务：HALFCHEETAH、ANT、HOPPER 和 WALKER2D。它们与 OpenAI Gym[29] 中的任务相似，但模拟器是开源的，且更难解决¹。在本节中，我们重点关注 SAC 算法，因为它将用于真实机器人，尽管我们在附录中报告了 PPO 等其他算法的结果。

为了评估平滑性，我们定义了一个连续代价函数，从 $(\mathbf{x}_0, \mathbf{a}_0)$ 开始，在 $(\mathbf{x}_T, \mathbf{a}_T)$ 处结束，其中 $\mathbf{x}_0$ 和 $\mathbf{x}_T$ 是环境的两个极端状态（例如，从左侧极限跳到另一个极限）。训练的连续性代价 C_{train} 是机器人磨

损的代理指标。我们比较以下配置的性能：(a) 无探索噪声，(b) 非结构化高斯噪声（原始 SAC 实现），(c) 相关噪声（Ornstein-Uhlenbeck 过程 [30]，参数为 $\sigma=0.2$ ，图中记为 OU 噪声），(d) 自适应参数噪声 [19] ($\sigma=0.2$)，(e) gSDE。为了将探索噪声与参数更新引起的噪声解耦，并更接近真实机器人设置，我们仅在每个回合结束时应用梯度更新。¹

<https://frama.link/PyBullet-harder-than-MuJoCo>

我们将预算固定为 100 万步，报告 10 次运行的平均得分以及训练期间的平均连续性代价及其标准误差。对于每次运行，我们每 10000 步在 20 个评估回合上测试学习到的策略，使用确定性控制器 $\mu(s_t)$ 。关于实现，我们使用修改版的 Stable-Baselines3[31] 以及 RL Zoo 训练框架 [32]。我们调整超参数所遵循的方法及其细节可在附录中找到。我们用于运行实验和调整超参数的代码可在补充材料中找到。

Algorithm	HALFCHEETAH		ANT		HOPPER		WALKER2D	
SAC	Return $\uparrow$	$C_{\text{train}} \downarrow$	Return $\uparrow$	$C_{\text{train}} \downarrow$	Return $\uparrow$	$C_{\text{train}} \downarrow$	Return $\uparrow$	$C_{\text{train}} \downarrow$
w/o noise	2562 $\pm$ 102	2.6 $\pm$ 0.1	2600 $\pm$ 364	2.0 $\pm$ 0.2	1661 $\pm$ 270	1.8 $\pm$ 0.1	2216 $\pm$ 40	1.8 $\pm$ 0.1
w/ unstructured	2994 $\pm$ 89	4.8 $\pm$ 0.2	3394 $\pm$ 64	5.1 $\pm$ 0.1	2434 $\pm$ 190	3.6 $\pm$ 0.1	2225 $\pm$ 35	3.6 $\pm$ 0.1
w/ OU noise	2692 $\pm$ 68	2.9 $\pm$ 0.1	2849 $\pm$ 267	2.3 $\pm$ 0.0	2200 $\pm$ 53	2.1 $\pm$ 0.1	2089 $\pm$ 25	2.0 $\pm$ 0.0
w/ param noise	2834 $\pm$ 54	2.9 $\pm$ 0.1	3294 $\pm$ 55	2.1 $\pm$ 0.1	1685 $\pm$ 279	2.2 $\pm$ 0.1	2294 $\pm$ 40	1.8 $\pm$ 0.1
w/ gSDE-8	2850 $\pm$ 73	4.1 $\pm$ 0.2	3459 $\pm$ 52	3.9 $\pm$ 0.2	2646 $\pm$ 45	2.4 $\pm$ 0.1	2341 $\pm$ 45	2.5 $\pm$ 0.1
w/ gSDE-64	2970 $\pm$ 132	3.5 $\pm$ 0.1	3160 $\pm$ 184	3.5 $\pm$ 0.1	2476 $\pm$ 99	2.0 $\pm$ 0.1	2324 $\pm$ 39	2.3 $\pm$ 0.1
w/ gSDE-episodic	2741 $\pm$ 115	3.1 $\pm$ 0.2	3044 $\pm$ 106	2.6 $\pm$ 0.1	2503 $\pm$ 80	1.8 $\pm$ 0.1	2267 $\pm$ 34	2.2 $\pm$ 0.1

表 1: 在 PyBullet 环境中，SAC 采用不同类型探索的详细回报和连续性代价结果。我们报告了 100 万步 10 次运行的均值和标准误差。对于每个基准，当差异具有统计显著性时，我们突出显示具有最佳均值的方法的结果。

结果 和图 2 展示了在 PyBullet 任务上的结果以及连续性与性能之间的权衡。在没有任何噪声的情况下（图中“无噪声”），SAC 仍然能够借助成形奖励部分解决这些任务，但其结果方差最高。尽管相关噪声和参数噪声在训练期间产生的连续性代价较低，但这是以性能为代价的。gSDE 通过利用噪声重复参数，能够在非结构化探索与相关噪声之间取得良好的折中。gSDE-8（每 8 步采样一次噪声）甚至在训练时以更低的连续性代价实现了更好的性能。这种行为正是真实机器人训练所期望的：我们必须在训练时尽量减少磨损，同时在测试时仍能获得良好性能。

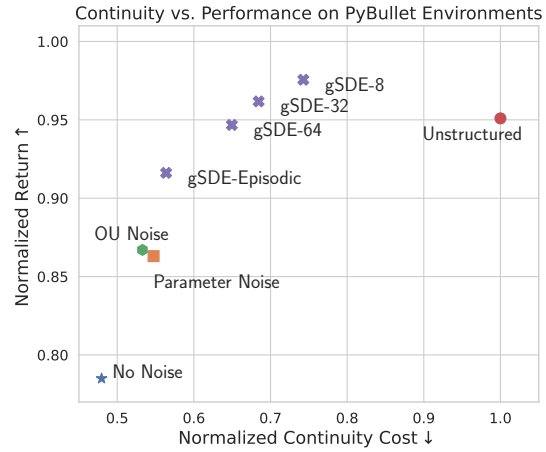


图 2: 在 4 个 PyBullet 任务上，SAC 采用不同类型探索的归一化回报和连续性代价。gSDE 在性能和平滑性之间提供了折衷。

4.2 与原始 SDE 的比较

在本节中，我们研究了对原始 SDE 所提出的修改的贡献：每 n 步采样一次探索函数参数，并使用策略特征作为噪声函数的输入。

采样间隔

gSDE is a n 其中 n 允许在非结构化探索 $n = 1$ 与原始 SDE 的逐回合公式之间进行插值。这种插值使得在训练时能够在性能与平滑性之间取得折中（参见表 1 和图 2）。图 3b 展示了该参数对于 PPO 在 WALKER2D 任务上的重要性。如果采样间隔过大，智能体在长回合中将无法充分探索。另一方面，当采样频率较高 $n \approx 1$ 时，第 1 节中提到的问题就会出现。

策略特征作为输入 图 3a 展示了改变 SAC 和 PPO 探索函数输入的效果。尽管因任务而异，使用策略特征（图中“潜在”）通常有益，尤其对 PPO 而言。此外，它需要的调参更少，且无需归一化，因为它仅依赖于策略网络架构。此处，PyBullet 任务维度较低，状态空间大小处于同一数量级，因此无需针对每个任务进行精细调参。依赖特征还允许直接从像素学习，这在原始公式中是不可能的。

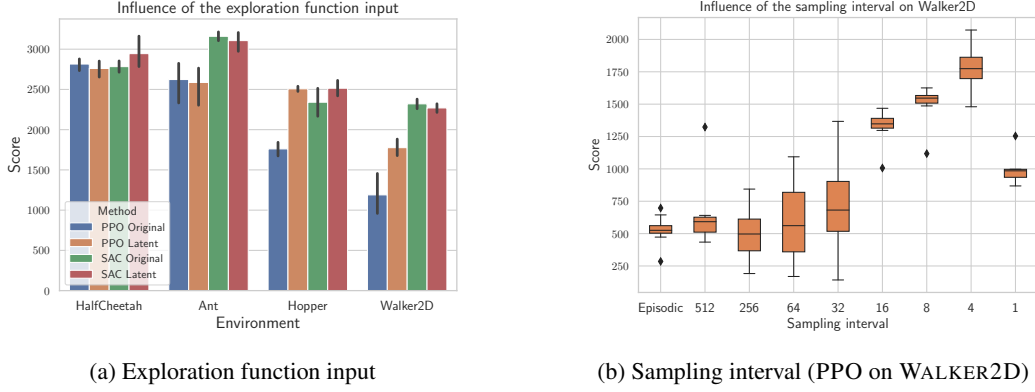
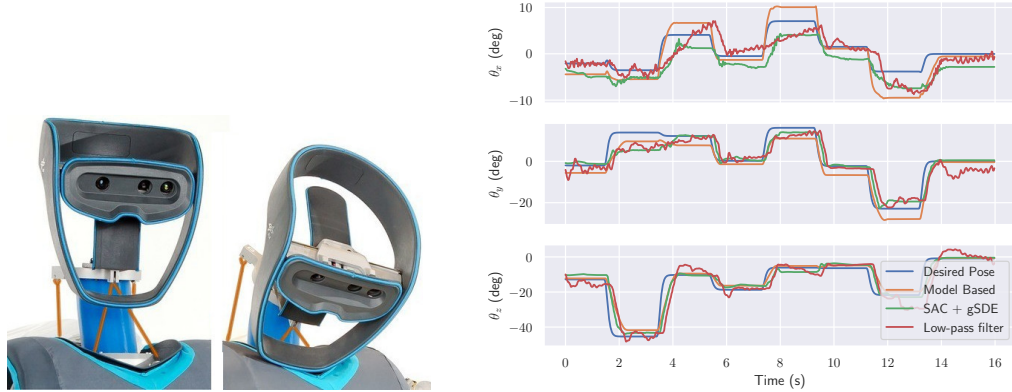


图 3: gSDE 修改相对于原始 SDE 在 PyBullet 任务上的影响。(a) 探索函数输入 $\epsilon(s; \theta_\epsilon)$ 对 SAC 和 PPO 的影响: 使用策略的潜在特征 $\mathbf{z}_\mu$ (潜在) 通常优于使用状态 $\mathbf{s}$ (原始)。(b) 噪声函数参数的采样频率对使用 gSDE 的 PPO 至关重要。

与原始 SDE 相比, 所提出的两项修改均有利于性能, 其中噪声采样间隔 n 影响最大。幸运的是, 如表 1 和图 2 所示, 对于 SAC 而言, 该间隔可以相当自由地选择。在附录中, 我们提供了额外的消融实验, 表明 gSDE 对初始探索方差的选择具有鲁棒性。

4.3 学习控制腱驱动弹性机器人



(a) Tendon-driven elastic continuum neck in a humanoid robot (b) Model-Based controller vs RL controller on the real robot

图 4: (a) 实验中使用的腱驱动机器人 [33]。腱以橙色高亮显示。(b) 基于模型的控制与强化学习智能体在评估轨迹上表现相似。

实验设置 为评估 gSDE 的实用性, 我们将其应用于真实系统。任务是控制一个腱驱动弹性连续体颈部 [33] (见图 4a) 到达给定的目标姿态。控制此类软体机器人具有挑战性, 原因在于非线性腱耦合以及需要精确建模的结构变形。该建模计算成本高昂 [34, 35], 且需要可能在实际物理系统中不成立的假设。

该系统欠驱动 (仅有 4 根肌腱), 因此期望位姿为 4 维向量: 3 个旋转角 $\theta_x, \theta_y, \theta_z$ 和 1 个位置角 x 。输入为 16 维向量, 由以下部分组成: 测量的肌腱长度 (4 维)、当前肌腱力 (4 维)、当前位姿 (4 维) 和目标位姿 (4 维)。奖励为到期望方向的负测地距离与到期望位置的负欧氏距离的加权和。权重选择使得两个分量具有相同的量级。我们还添加了一个小的连续性代价以减少

最终策略中的振荡。动作空间由期望的肌腱力增量组成，限制在 5 牛以内。出于安全考虑，肌腱力被裁剪在 10 牛以下和 40 牛以上。一个回合在智能体到达期望位姿或 5 秒超时后终止。如果期望位姿在位置 10 毫米和方向 5° 的阈值内到达，则该回合被视为成功。智能体以 30 赫兹的频率控制肌腱力，而机器人上的比例微分控制器以 3 千赫兹的频率监控电机电流。梯度更新直接在四核笔记本电脑上每个回合后进行。

Results We first ran the unstructured exploration on the robot, but had to stop the experiment early: the high-frequency noise in the command was damaging the tendons and would have broken them due to their friction on the bearings. Therefore, as a baseline, we trained a policy using SAC with a hand-crafted action smoothing (2 Hz cutoff Butterworth low-pass filter) for two hours. Then, we trained a controller using SAC with gSDE for the same duration. We compare both learned controllers to an existing model-based controller (passivity-based approach) presented in [34, 35] using a pre-defined trajectory (cf. Fig. 4b). On the evaluation trajectory, the controllers are equally precise (cf. Table 3): the mean error in orientation is below 3° and the one in position below 3 mm. However, the policy trained with the low-pass filter is much more jittery than the two others. We quantify this jitter as the mean absolute difference between two timesteps, denoted as *continuity cost* in Table 3.

	Unstructured noise	gSDE	Low-pass filter	Model-Based
Position error (mm)	N/A	2.65 +/- 1.6	1.98 +/- 1.7	1.32 +/- 1.2
Orientation error (deg)	N/A	2.85 +/- 2.9	3.53 +/- 4.0	2.90 +/- 2.8
Continuity cost (deg)	N/A	0.20 +/- 0.04	0.38 +/- 0.07	0.16 +/- 0.04

表 2: 评估轨迹上位置、方向和平均连续性代价的误差比较。当差异显著时，我们突出显示最佳方法。基于模型和学习的控制器产生可比较的结果，但使用低通滤波器训练的策略具有更高的连续性代价。

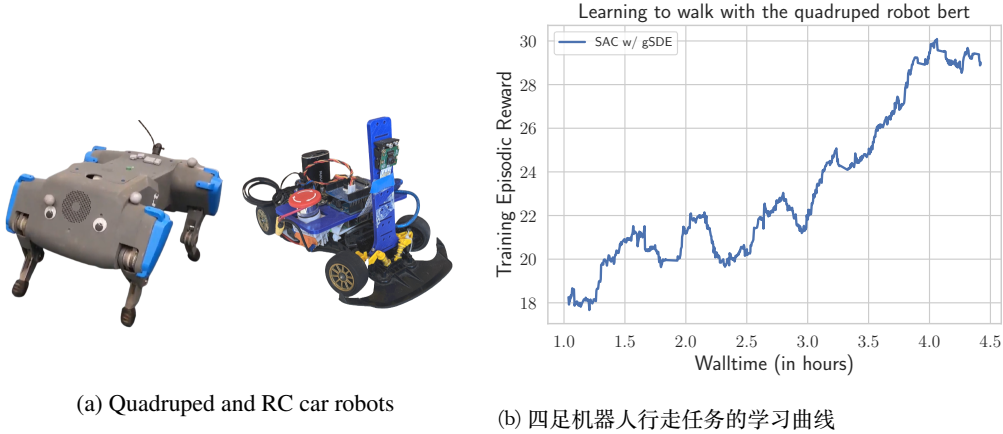


图 5: 使用带 gSDE 的 SAC 直接在现实世界中成功训练的其他机器人。

额外的真实机器人实验 我们还成功地将带 gSDE 的 SAC 应用于两个额外的真实机器人任务（见图 5a）：训练弹性四足机器人行走（学习曲线见图 5b）以及学习使用遥控车在赛道上行驶。两个实验完全在真实机器人上完成，没有使用仿真或滤波器。我们在补充材料中提供了训练控制器的视频。

5 相关工作

探索是强化学习中的一个关键主题 [36]。在离散情形下，探索已被广泛研究，而近期大多数论文仍聚焦于离散动作 [37, 38]。

若干工作通过将非结构化探索替换为相关噪声，来解决连续控制中的非结构化探索问题。Korenkevych 等人 [15] 使用自回归过程，并引入两个变量来控制探索的平滑性。类似地，van Hoof 等人 [27] 利用时间相干性参数在基于步长或基于回合的探索之间进行插值，并借助马尔可夫链来关联噪声。这种平滑噪声的代价是：它需要历史信息，从而改变了问题定义。

在参数空间中进行探索 [10, 39, 11, 12, 40] 是一种正交方法，同样解决了非结构化探索的某些问题。该方法已成功应用于真实机器人，但依赖于运动基元 [41, 12]，这需要专家知识。Plappert 等人 [19] 通过在动作空间中定义距离并应用层归一化来处理高维空间，从而将参数探索适配到深度强化学习中。

基于种群的算法，如进化策略或遗传算法，也在参数空间中进行探索。得益于大规模并行化，它们在训练时间上被证明可与强化学习相竞争 [42]，但代价是样本效率低下。为解决这一问题，近期工作 [20] 提出将进化策略探索与强化学习梯度更新相结合。这种结合虽然强大，但不幸的是增加了大量超参数和不可忽略的计算开销。

获得平滑控制对于真实机器人至关重要，但通常被深度强化学习社区所忽视。最近，Mysore 等人 [14] 在强化学习算法中引入了连续性和平滑性损失。他们的方法能有效获得平滑控制器，从而减少真实机器人测试时的能耗。然而，该方法并未解决训练时平滑探索的问题，因此其训练仅限于仿真环境。

6 结论

本文指出了深度强化学习算法在连续控制中因非结构化探索而产生的若干问题。由于这些问题，这些算法无法直接应用于真实世界机器人的学习。

为解决上述问题，本文通过扩展原始公式，将状态依赖探索 (State-Dependent Exploration) 适配到深度强化学习算法中：每 n 步采样一次噪声，并以学习到的特征替代探索函数的输入。该广义版本 (gSDE) 为无结构高斯探索提供了一种简单且高效的替代方案。gSDE 在多个连续控制基准上取得了极具竞争力的结果，同时减少了训练过程中的磨损。本文还通过消融实验研究了每项修改的贡献：噪声采样间隔的影响最大，能够在性能与平滑性之间取得折中。本文提出的探索策略与 SAC 结合后，对超参数选择具有鲁棒性，因此适用于机器人应用。为验证这一点，本文成功地将结合 gSDE 的 SAC 直接应用于三种不同的机器人。

尽管仿真到现实 (sim2real) 方法取得了很大进展，本文认为应投入更多精力直接在真实系统上进行学习，即使这在安全性和学习时长方面带来挑战。本文旨在向这一目标迈进一步，并希望重新激发对可直接应用于真实机器人的探索方法的研究兴趣。

参考文献

- [1] T. Rückstieß, M. Felder, and J. Schmidhuber. State-dependent exploration for policy gradient methods. In *Joint European Conference on Machine Learning and Knowledge Discovery in Databases*, pages 234–249. Springer, 2008.
- [2] N. J. Nilsson. Shakey the robot. Technical Report 323, Artificial Intelligence Center, SRI International, Menlo Park, CA, USA, 1984. URL <http://www.ai.sri.com/shakey/>.

- [3] Y. Duan, X. Chen, R. Houthoof, J. Schulman, and P. Abbeel. Benchmarking deep reinforcement learning for continuous control. In *International Conference on Machine Learning*, pages 1329–1338, 2016.
- [4] M. Andrychowicz, B. Baker, M. Chociej, R. Jozefowicz, B. McGrew, J. Pachocki, A. Petron, M. Plappert, G. Powell, A. Ray, et al. Learning dexterous in-hand manipulation. *arXiv preprint arXiv:1808.00177*, 2018.
- [5] S. Fujimoto, H. van Hoof, and D. Meger. Addressing function approximation error in actor-critic methods. *arXiv preprint arXiv:1802.09477*, 2018.
- [6] X. B. Peng, P. Abbeel, S. Levine, and M. van de Panne. Deepmimic: Example-guided deep reinforcement learning of physics-based character skills. *ACM Transactions on Graphics (TOG)*, 37(4):143, 2018.
- [7] J. Hwangbo, J. Lee, A. Dosovitskiy, D. Bellicoso, V. Tsounis, V. Koltun, and M. Hutter. Learning agile and dynamic motor skills for legged robots. *arXiv preprint arXiv:1901.08652*, 2019.
- [8] T. Haarnoja, A. Zhou, K. Hartikainen, G. Tucker, S. Ha, J. Tan, V. Kumar, H. Zhu, A. Gupta, P. Abbeel, et al. Soft actor-critic algorithms and applications. *arXiv preprint arXiv:1812.05905*, 2018.
- [9] A. Kendall, J. Hawke, D. Janz, P. Mazur, D. Reda, J.-M. Allen, V.-D. Lam, A. Bewley, and A. Shah. Learning to drive in a day. In *2019 International Conference on Robotics and Automation (ICRA)*, pages 8248–8254. IEEE, 2019.
- [10] J. Kober and J. R. Peters. Policy search for motor primitives in robotics. In *Advances in neural information processing systems*, pages 849–856, 2009.
- [11] T. Rückstieß, F. Sehnke, T. Schaul, D. Wierstra, Y. Sun, and J. Schmidhuber. Exploring parameter space in reinforcement learning. *Paladyn, Journal of Behavioral Robotics*, 1(1): 14–24, 2010.
- [12] F. Stulp and O. Sigaud. Robot skill learning: From reinforcement learning to evolution strategies. *Paladyn, Journal of Behavioral Robotics*, 4(1):49–61, 2013.
- [13] M. P. Deisenroth, G. Neumann, J. Peters, et al. A survey on policy search for robotics. *Foundations and Trends® in Robotics*, 2(1–2):1–142, 2013.
- [14] S. Mysore, B. Mabsout, R. Mancuso, and K. Saenko. Regularizing action policies for smooth control with reinforcement learning. 2021.
- [15] D. Korenkevych, A. R. Mahmood, G. Vasan, and J. Bergstra. Autoregressive policies for continuous control deep reinforcement learning. *arXiv preprint arXiv:1903.11524*, 2019.
- [16] T. Haarnoja, S. Ha, A. Zhou, J. Tan, G. Tucker, and S. Levine. Learning to walk via deep reinforcement learning. *arXiv preprint arXiv:1812.11103*, 2018.
- [17] S. Ha, P. Xu, Z. Tan, S. Levine, and J. Tan. Learning to walk in the real world with minimal human effort. *arXiv preprint arXiv:2002.08550*, 02 2020.
- [18] M. Neunert, A. Abdolmaleki, M. Wulfmeier, T. Lampe, J. T. Springenberg, R. Hafner, F. Romano, J. Buchli, N. Heess, and M. Riedmiller. Continuous-discrete reinforcement learning for hybrid control in robotics. *arXiv preprint arXiv:2001.00449*, 2020.
- [19] M. Plappert, R. Houthoof, P. Dhariwal, S. Sidor, R. Y. Chen, X. Chen, T. Asfour, P. Abbeel, and M. Andrychowicz. Parameter space noise for exploration. *arXiv preprint arXiv:1706.01905*, 2017.
- [20] A. Pourchot and O. Sigaud. Cem-rl: Combining evolutionary and gradient-based methods for policy search. *arXiv preprint arXiv:1810.01222*, 2018.
- [21] J. Schulman, S. Levine, P. Abbeel, M. Jordan, and P. Moritz. Trust region policy optimization. In *International conference on machine learning*, pages 1889–1897, 2015.

- [22] T. P. Lillicrap, J. J. Hunt, A. Pritzel, N. Heess, T. Erez, Y. Tassa, D. Silver, and D. Wierstra. Continuous control with deep reinforcement learning. *arXiv preprint arXiv:1509.02971*, 2015.
- [23] V. Mnih, A. P. Badia, M. Mirza, A. Graves, T. Lillicrap, T. Harley, D. Silver, and K. Kavukcuoglu. Asynchronous methods for deep reinforcement learning. In *International conference on machine learning*, pages 1928–1937, 2016.
- [24] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.
- [25] T. Haarnoja, H. Tang, P. Abbeel, and S. Levine. Reinforcement learning with deep energy-based policies. In *Proceedings of the 34th International Conference on Machine Learning-Volume 70*, pages 1352–1361. JMLR. org, 2017.
- [26] R. J. Williams. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine learning*, 8(3-4):229–256, 1992.
- [27] H. van Hoof, D. Tanneberg, and J. Peters. Generalized exploration in policy search. *Machine Learning*, 106(9-10):1705–1724, oct 2017. Special Issue of the ECML PKDD 2017 Journal Track.
- [28] E. Coumans and Y. Bai. Pybullet, a python module for physics simulation for games, robotics and machine learning. <http://pybullet.org>, 2016–2019.
- [29] G. Brockman, V. Cheung, L. Pettersson, J. Schneider, J. Schulman, J. Tang, and W. Zaremba. Openai gym. *arXiv preprint arXiv:1606.01540*, 2016.
- [30] G. E. Uhlenbeck and L. S. Ornstein. On the theory of the brownian motion. *Physical review*, 36(5):823, 1930.
- [31] A. Raffin, A. Hill, M. Ernestus, A. Gleave, A. Kanervisto, and N. Dormann. Stable baselines3. <https://github.com/DLR-RM/stable-baselines3>, 2019.
- [32] A. Raffin. RL baselines3 zoo. <https://github.com/DLR-RM/rl-baselines3-zoo>, 2020.
- [33] J. Reinecke, B. Deutschmann, and D. Fehrenbach. A structurally flexible humanoid spine based on a tendon-driven elastic continuum. In *2016 IEEE International Conference on Robotics and Automation (ICRA)*, pages 4714–4721. IEEE, 2016.
- [34] B. Deutschmann, A. Dietrich, and C. Ott. Position control of an underactuated continuum mechanism using a reduced nonlinear model. In *2017 IEEE 56th Annual Conference on Decision and Control (CDC)*, pages 5223–5230. IEEE, 2017.
- [35] B. Deutschmann, M. Chalon, J. Reinecke, M. Maier, and C. Ott. Six-dof pose estimation for a tendon-driven continuum mechanism without a deformation model. *IEEE Robotics and Automation Letters*, 4(4):3425–3432, 2019.
- [36] R. S. Sutton and A. G. Barto. *Reinforcement learning: An introduction*. MIT press, 2018.
- [37] I. Osband, C. Blundell, A. Pritzel, and B. Van Roy. Deep exploration via bootstrapped dqn. In *Advances in neural information processing systems*, pages 4026–4034, 2016.
- [38] I. Osband, J. Aslanides, and A. Cassirer. Randomized prior functions for deep reinforcement learning. In *Advances in Neural Information Processing Systems*, pages 8617–8629, 2018.
- [39] F. Sehnke, C. Osendorfer, T. Rückstieß, A. Graves, J. Peters, and J. Schmidhuber. Parameter-exploring policy gradients. *Neural Networks*, 23(4):551–559, 2010.
- [40] O. Sigaud and F. Stulp. Policy search in continuous action domains: an overview. *Neural Networks*, 2019.
- [41] J. Peters and S. Schaal. Reinforcement learning of motor skills with policy gradients. *Neural networks*, 21(4):682–697, 2008.

- [42] F. Such, V. Madhavan, E. Conti, J. Lehman, K. Stanley, and J. Clune. Deep neuroevolution: Genetic algorithms are a competitive alternative for training deep neural networks for reinforcement learning. *arXiv preprint arXiv:1712.06567*, 12 2017.
- [43] J. Schulman, P. Moritz, S. Levine, M. Jordan, and P. Abbeel. High-dimensional continuous control using generalized advantage estimation. *arXiv preprint arXiv:1506.02438*, 2015.
- [44] D. Silver, G. Lever, N. Heess, T. Degris, D. Wierstra, and M. Riedmiller. Deterministic policy gradient algorithms. In *Proceedings of the 31st International Conference on International Conference on Machine Learning - Volume 32*, ICML’14, page I–387–I–395. JMLR.org, 2014.
- [45] V. Mnih, K. Kavukcuoglu, D. Silver, A. Graves, I. Antonoglou, D. Wierstra, and M. Riedmiller. Playing atari with deep reinforcement learning. *arXiv preprint arXiv:1312.5602*, 2013.
- [46] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. *arXiv preprint arXiv:1801.01290*, 2018.
- [47] A. Raffin and R. Sokolov. Learning to drive smoothly in minutes. <https://github.com/araffin/learning-to-drive-in-5-minutes/>, 2019.
- [48] A. Hill, A. Raffin, M. Ernestus, A. Gleave, A. Kanervisto, R. Traore, P. Dhariwal, C. Hesse, O. Klimov, A. Nichol, M. Plappert, A. Radford, J. Schulman, S. Sidor, and Y. Wu. Stable baselines. <https://github.com/hill-a/stable-baselines>, 2018.
- [49] L. Engstrom, A. Ilyas, S. Santurkar, D. Tsipras, F. Janoos, L. Rudolph, and A. Madry. Implementation matters in deep {rl}: A case study on {ppo} and {trpo}. In *International Conference on Learning Representations*, 2020. URL <https://openreview.net/forum?id=r1etN1rtPB>.
- [50] A. Raffin. Rl baselines zoo. <https://github.com/araffin/rl-baselines-zoo>, 2018.
- [51] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama. Optuna: A next-generation hyperparameter optimization framework. In *Proceedings of the 25rd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2019.
- [52] F. Pardo, A. Tavakoli, V. Levдик, and P. Kormushev. Time limits in reinforcement learning. *arXiv preprint arXiv:1712.00378*, 2017.
- [53] A. Rajeswaran, K. Lowrey, E. V. Todorov, and S. M. Kakade. Towards generalization and simplicity in continuous control. In *Advances in Neural Information Processing Systems*, pages 6550–6561, 2017.
- [54] D. P. Kingma and J. Ba. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2014.

A 补充材料

A.1 状态依赖探索

在线性情况下，即使用线性策略和噪声矩阵时，参数空间探索与状态依赖探索（SDE）是等价的：

$$\begin{aligned}\mathbf{a}_t &= \mu(\mathbf{s}_t; \theta_\mu) + \epsilon(\mathbf{s}_t; \theta_\epsilon), & \theta_\epsilon &\sim \mathcal{N}(0, \sigma^2) \\ &= \theta_\mu \mathbf{s}_t + \theta_\epsilon \mathbf{s}_t \\ &= (\theta_\mu + \theta_\epsilon) \mathbf{s}_t\end{aligned}$$

由于已知策略分布，可以求得对数似然 $\log \pi(\mathbf{a}|\mathbf{s})$ 关于方差 σ 的导数：

$$\frac{\partial \log \pi(\mathbf{a}|\mathbf{s})}{\partial \sigma_{ij}} = \sum_k \frac{\partial \log \pi_k(\mathbf{a}_k|\mathbf{s})}{\partial \hat{\sigma}_j} \frac{\partial \hat{\sigma}_j}{\partial \sigma_{ij}} \quad (7)$$

$$= \frac{\partial \log \pi_j(\mathbf{a}_j|\mathbf{s})}{\partial \hat{\sigma}_j} \frac{\partial \hat{\sigma}_j}{\partial \sigma_{ij}} \quad (8)$$

$$= \frac{(\mathbf{a}_j - \mu_j)^2 - \hat{\sigma}_j^2}{\hat{\sigma}_j^3} \frac{\mathbf{s}_i^2 \sigma_{ij}}{\hat{\sigma}_j} \quad (9)$$

这可以方便地代入似然比梯度估计器 [26]，从而在训练过程中自适应 σ 。因此，状态依赖探索（SDE）与标准策略梯度方法兼容，同时解决了无结构探索的大部分缺陷。

A.2 算法

本节简要介绍本文所使用的算法。这些算法在无模型强化学习的连续控制任务中，无论是样本效率还是实际运行时间方面，均代表了当前最先进的方法。

A2C **A2C** 是异步优势演员-评论家（Asynchronous Advantage Actor-Critic, A3C）[23] 的同步版本。它是一种演员-评论家方法，利用 n 步的并行采样来更新策略。它依赖 REINFORCE [26] 估计器来计算梯度。**A2C** 速度快，但样本效率不高。

PPO **A2C** 的梯度更新无法防止大幅变化，这会导致性能急剧下降。为解决这一问题，信赖域策略优化（Trust Region Policy Optimization, TRPO）[21] 在策略参数空间中引入信赖域，将其表述为一个约束优化问题：在更新策略的同时，保持与旧策略的 KL 散度接近。其后续方法近端策略优化（Proximal Policy Optimization, PPO）[24] 通过使用重要性比率对目标进行裁剪，放宽了约束（该约束需要昂贵的共轭梯度步骤）。PPO 同样使用多个工作进程（与 A2C 相同）以及广义优势估计（Generalized Advantage Estimation, GAE）[43] 来计算优势。

TD3 深度确定性策略梯度（Deep Deterministic Policy Gradient, DDPG）[22] 将确定性策略梯度算法 [44] 与深度 Q 网络（Deep Q-Network, DQN）[45] 的改进相结合：使用经验回放缓冲区和目标网络来稳定训练。其直接后继方法双延迟深度确定性策略梯度（Twin Delayed DDPG, **TD3**）[5] 引入了三个主要技巧来解决函数逼近带来的问题：裁剪双 Q 学习（以减少 Q 值函数的过高估计）、延迟策略更新（使价值函数先收敛）以及目标策略平滑（以防止过拟合）。由于策略是确定性的，DDPG 和 **TD3** 依赖外部噪声进行探索。

SAC 软演员-评论家（Soft Actor-Critic）[25] 是软 Q 学习（Soft Q-Learning, SQL）[46] 的后继方法，优化最大熵目标，这与经典强化学习目标略有不同：

$$J(\pi) = \sum_{t=0}^T \mathbb{E}_{(\mathbf{s}_t, \mathbf{a}_t) \sim \rho_\pi} [r(\mathbf{s}_t, \mathbf{a}_t) + \alpha \mathcal{H}(\pi(\cdot | \mathbf{s}_t))]. \quad (10)$$

其中 $\mathcal{H}$ 为策略熵， α 为熵温度系数，用于在两个目标之间进行权衡。

SAC 使用压缩高斯分布学习随机策略，并借鉴了 TD3 中的截断双 Q 学习技巧。在其最新迭代 [8] 中，SAC 自动调整熵系数 α ，从而无需手动调节这一关键超参数。

哪种算法适用于机器人？ A2C 和 PPO 均为同策略算法，易于并行化，训练时间相对较短。另一方面，SAC 和 TD3 为异策略算法，在单个工作进程上运行，但样本效率远高于前两种方法，仅需少量样本即可达到相当的性能。

由于我们专注于机器人应用，通常无法拥有多台机器人，因此 TD3 和 SAC 成为首选方法。尽管 TD3 和 SAC 非常相似，但 SAC 将探索直接嵌入其目标函数中，使其更易于调节。在仿真实验中我们还发现，SAC 在广泛的超参数范围内均能正常工作。因此，我们采用该算法进行真实机器人实验和消融实验。

A.3 真实机器人实验

通用设置 对于每个真实机器人实验，为了提高最终控制器的平滑性并应对通信延迟（这会破坏马尔可夫假设），我们将前一次观测和上一次动作作为输入进行增广，并在奖励中加入较小的连续性代价。对于每个任务，我们将奖励函数分解为主要部分（我们希望实现的目标）和次要部分（如连续性代价等软约束）。每个奖励项均进行归一化处理，从而可以根据重要性轻松加权各个部分。与先前工作相比，我们使用与仿真任务完全相同的算法，因此避免了滤波器的使用。

学习控制弹性颈部。 回合在智能体达到期望姿态或 5 秒超时后终止，即每个回合的最大长度为 200 步。若在位置阈值 $10mm$ 和姿态阈值 $5deg$ 内达到期望姿态，则视为成功。

学习使用弹性四足机器人伯特行走。 智能体通过 Wi-Fi 接收关节角度、速度、力矩和惯性测量单元数据作为输入，并输出期望的绝对电机角度。主要奖励为行进距离，次要奖励为不同代价的加权和：航向代价、偏离中心线距离和连续性代价。借助跑步机，机器人的复位实现了半自动化。早停和机器人监控通过外部跟踪完成，但观测仅由机载传感器计算。若机器人跌倒、越界或 5 秒超时，回合终止。训练直接在真实机器人上进行，持续数天，总计约 8 小时的交互。

学习使用遥控车驾驶。 智能体接收机载摄像头图像作为输入，并输出期望的油门和转向角。特征使用预训练的自编码器计算，如 Raffin 和 Sokolov[47] 所述。主要奖励为生存奖励（安全驾驶员未干预）与指令油门之间的加权和。次要奖励仅为连续性代价。当安全驾驶员干预（碰撞）或 1 分钟超时后，回合终止。训练直接在机器人上进行，所需交互时间少于 30 分钟。

	SAC + gSDE	Model-Based [34]
Error in position (mm)	2.65 +/- 1.6	1.32 +/- 1.2
Error in orientation (deg)	2.85 +/- 2.9	2.90 +/- 2.8

表 3: 评估轨迹上位置和姿态平均误差的比较。基于模型的控制器与学习控制器结果相当。

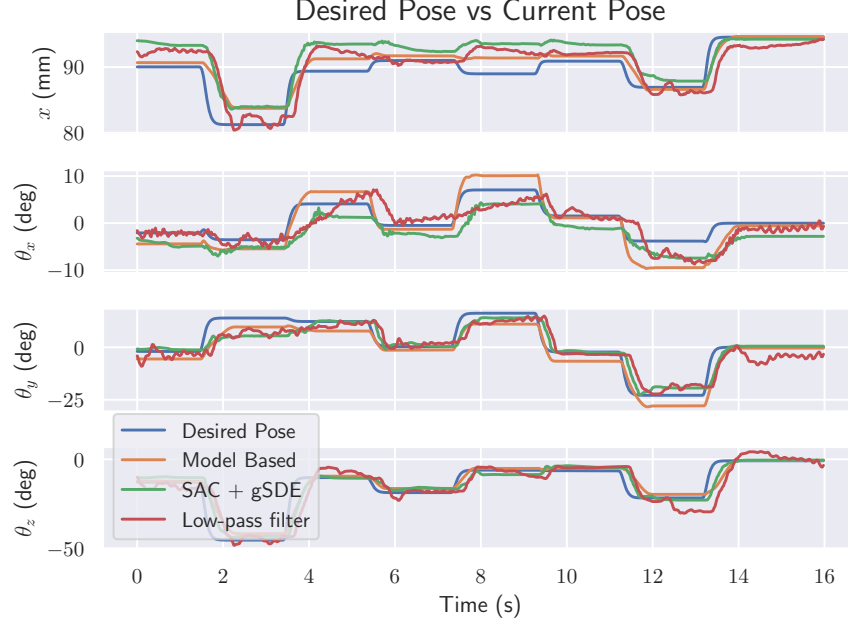


图 6: 基于模型的控制器与学习强化学习智能体在评估轨迹上的比较: 两者表现相似。

A.4 实现细节

我们使用了基于 PyTorch [31] 的 Stable-Baselines [48] 库, 并结合 RL Zoo 训练框架 [32]。对于使用独立高斯噪声的版本, 它采用了 PPO [49] 的常见实现技巧。

对于 SAC, 为确保数值稳定性, 我们将均值裁剪到 $[-2, 2]$ 范围内, 因为它会导致无穷值。在原始实现中, 改为对均值和标准差使用正则化 $\mathcal{L}_2$ 损失。带有 gSDE 的 SAC 算法在算法 1 中描述。

与原始 SDE 论文相比, 我们无需使用 expln 技巧 [1] 来避免 PyBullet 任务中的方差爆炸。然而, 我们发现在特定环境 (如 BipedalWalkerHardcore-v2) 中该技巧很有用。原始 SAC 实现会裁剪此方差。

Algorithm 1 Soft Actor-Critic with gSDE

```

Initialize parameters  $\theta_\mu, \theta_Q, \sigma, \alpha$ 
Initialize replay buffer  $\mathcal{D}$ 
for each iteration do
     $\theta_\epsilon \sim \mathcal{N}(0, \sigma^2)$  ▷ Sample noise function parameters
    for each environment step do
         $\mathbf{a}_t = \pi(\mathbf{s}_t) = \mu(\mathbf{s}_t; \theta_\mu) + \epsilon(\mathbf{s}_t; \theta_\epsilon)$  ▷ Get the noisy action
         $\mathbf{s}_{t+1} \sim p(\mathbf{s}_{t+1} | \mathbf{s}_t, \mathbf{a}_t)$  ▷ Step in the environment
         $\mathcal{D} \leftarrow \mathcal{D} \cup \{(\mathbf{s}_t, \mathbf{a}_t, r(\mathbf{s}_t, \mathbf{a}_t), \mathbf{s}_{t+1})\}$  ▷ Update the replay buffer
    end for
    for each gradient step do
         $\theta_\epsilon \sim \mathcal{N}(0, \sigma^2)$  ▷ Sample noise function parameters
        Sample a minibatch from the replay buffer  $\mathcal{D}$ 
        Update the entropy temperature  $\alpha$ 
        Update parameters using  $\nabla J_Q$  and  $\nabla J_\pi$  ▷ Update actor  $\mu$ , critic  $Q$  and noise variance  $\sigma$ 
        Update target networks
    end for
end for

```

A.5 学习曲线与附加结果

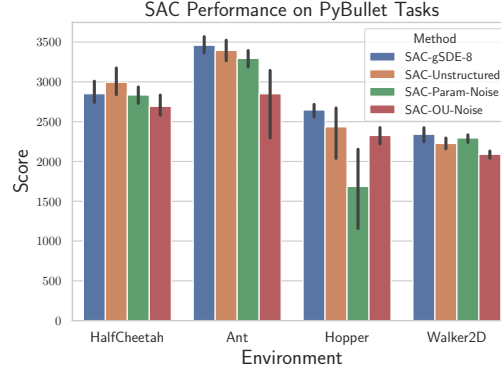


图 7: SAC 在 PyBullet 任务上采用不同探索类型的性能结果

A2C PPO 环境 gSDE 高斯 gSDE 高斯				
HALFCHEETAH	2028 +/- 107	1652 +/- 94	2760 +/- 52	2254 +/- 66
ANT	2560 +/- 45	1967 +/- 104	2587 +/- 133	2160 +/- 63
HOPPER	1448 +/- 163	1559 +/- 129	2508 +/- 16	1622 +/- 220
WALKER2D	694 +/- 73	443 +/- 59	1776 +/- 53	1238 +/- 75

表 4: A2C 和 PPO 在 4 个环境中使用 gSDE 和非结构化高斯探索的最终性能（越高越好）。我们报告了 10 次运行、每次 200 万步的平均值。对于每个基准，当差异具有统计显著性时，我们突出显示平均性能最佳的方法的结果。

SAC TD3 环境 gSDE 高斯 gSDE 高斯				
HALFCHEETAH	2850 +/- 73	2994 +/- 89	2578 +/- 44	2687 +/- 67
ANT	3459 +/- 52	3394 +/- 64	3267 +/- 34	2865 +/- 278
HOPPER	2646 +/- 45	2434 +/- 190	2353 +/- 78	2470 +/- 111
WALKER2D	2341 +/- 45	2225 +/- 35	1989 +/- 153	2106 +/- 67

表 5: SAC 和 TD3 在 4 个环境中使用 gSDE 和非结构化高斯探索的最终性能（越高越好）。我们报告 10 次运行、每次 100 万步的平均值。对于每个基准，当差异具有统计显著性时，我们突出显示平均性能最佳的方法的结果。

图 8 展示了 SAC 在不同类型探索噪声下的学习曲线。

图 9 和图 10 展示了离策略和同策略算法在四个 PyBullet 任务上使用 gSDE 或非结构化高斯探索的学习曲线。

A.6 消融实验：附加图表

图 11 展示了其余 PyBullet 任务上的消融实验。结果表明 SAC 对初始探索方差具有鲁棒性，而 PPO 的结果高度依赖于噪声采样间隔。

并行采样 图 12a 展示了 PPO 中每个工作进程采样一组噪声参数的效果。这种修改改善了每个任务的性能，因为它允许更多样化的探索。尽管不太显著，我们在 PyBullet 环境上的 A2C 也观察到相同的结果（参见图 12b）。因此，利用并行工作进程可以改善探索和最终性能。

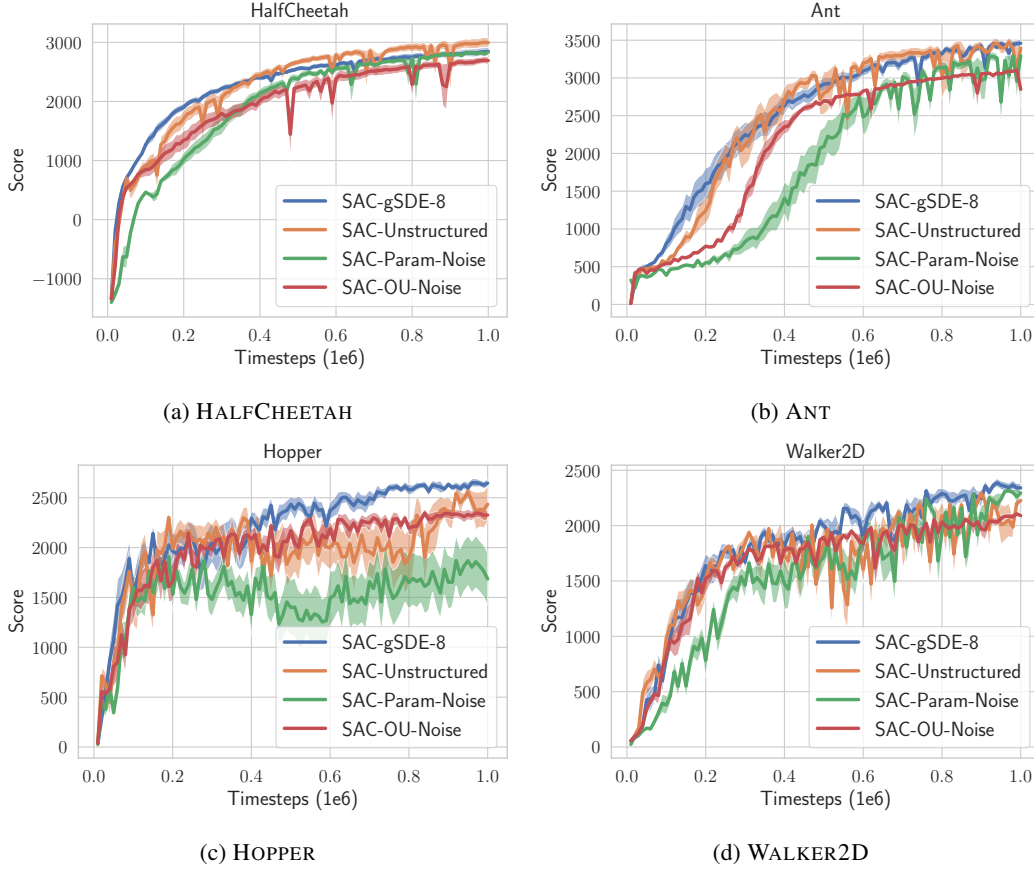


图 8: 不同探索类型下 SAC 在 PyBullet 任务上的学习曲线。曲线表示 10 次运行、每次 100 万步的平均值。

A.7 超参数优化

非结构化探索的 PPO 和 TD3 超参数沿用原始论文 [24, 5] 中的设置。对于 SAC，针对 gSDE 优化的超参数表现优于 Haarnoja 等人 [46] 的设置，因此我们将其保留用于其他探索类型，以确保公平比较。Mnih 等人 [23] 未提供 A2C 的超参数，因此我们采用 Raffin[50] 中调优后的设置。

为了调优超参数，我们使用 Optuna[51] 库中的 TPE 采样器和中位数剪枝器。在 HALF CHEETAH 环境中，我们提供 500 个候选方案，每个方案最多运行 $3 \cdot 10^5$ 个时间步。随后手动调整部分超参数（例如增大回放缓冲区大小）以提高算法的稳定性。

A.8 超参数

对于所有带时间限制的实验，如 [3, 52, 53, 48] 所述，我们在观测中加入时间特征（回合结束前的剩余时间），以避免破坏马尔可夫假设。该特征对性能影响显著，如图 13b 所示。

图 13a 展示了 SAC 在 PyBullet 任务上网络架构的影响。更大的网络通常能获得更好的结果，但超过一定复杂度后增益甚微（此处为每层 256 个单元的两层神经网络）。

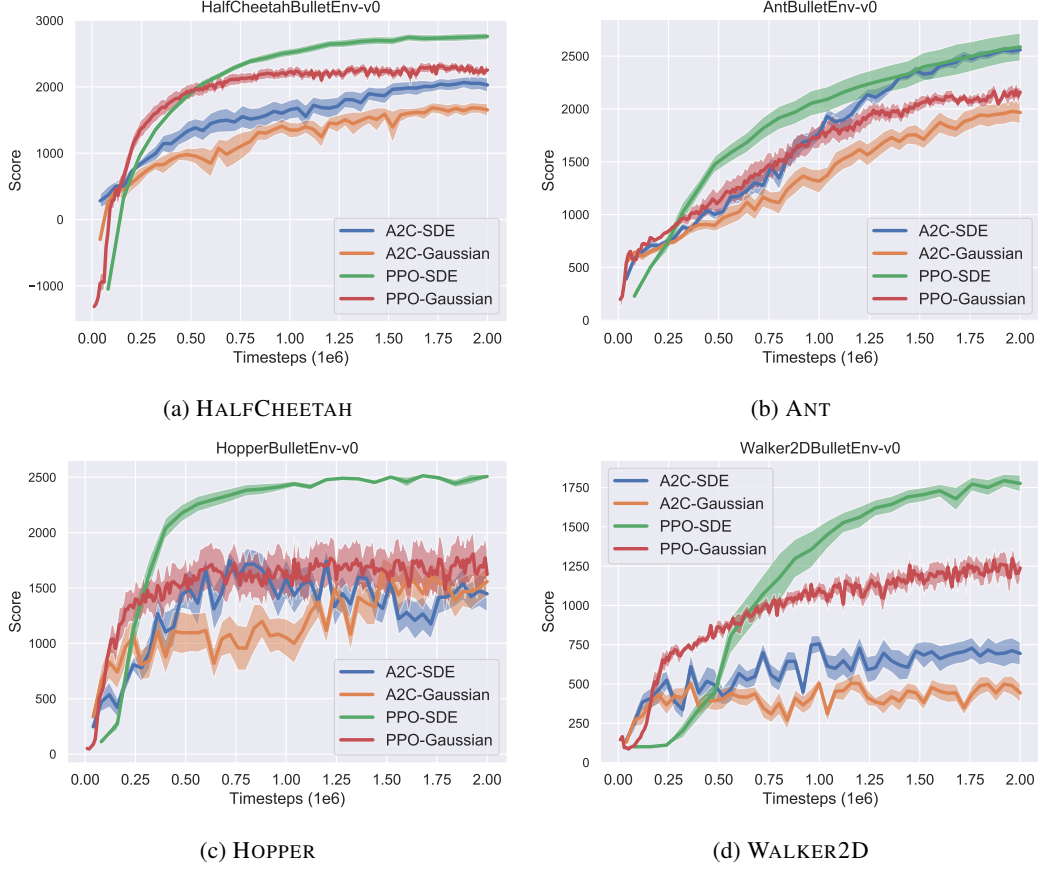


图 9: 在线策略算法在 PyBullet 任务上的学习曲线。曲线表示 10 次运行、每次 200 万步的平均值。

表 6: SAC 超参数

参数值	
<i>Shared</i>	
optimizer	Adam [54]
learning rate	$7.3 \cdot 10^{-4}$
learning rate schedule	constant
discount (γ)	0.98
replay buffer size	$3 \cdot 10^5$
number of hidden layers (all networks)	2
number of hidden units per layer	[400, 300]
number of samples per minibatch	256
non-linearity	<i>ReLU</i>
entropy coefficient (α)	auto
target entropy	$-\dim(\mathcal{A})$
target smoothing coefficient (τ)	0.02
train frequency	episodic
warm-up steps	10 000
normalization	None
<i>gSDE</i>	
initial log σ	-3
gSDE sample frequency	8

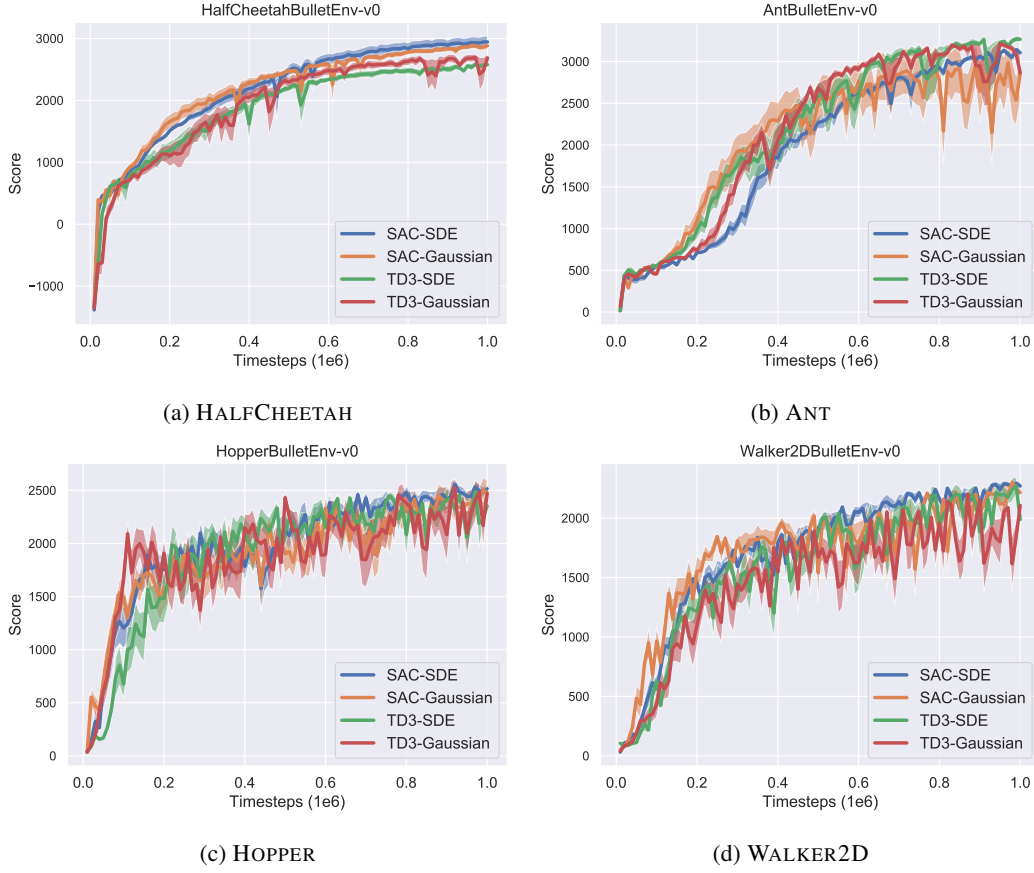


图 10: PyBullet 任务上离线策略算法的学习曲线。曲线表示 10 次运行、每次 100 万步的均值。

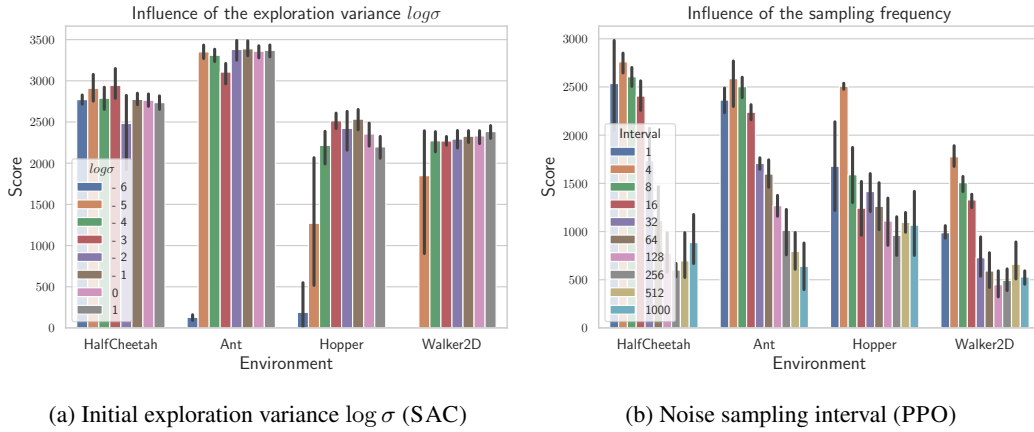
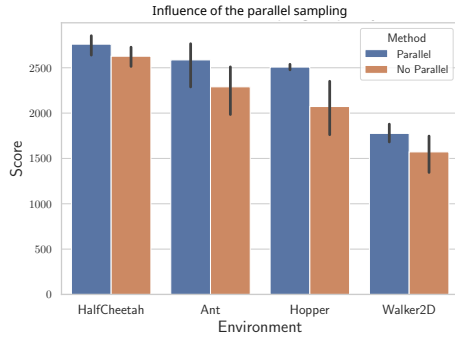


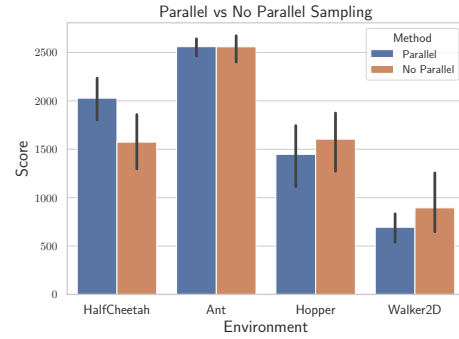
图 11: PyBullet 任务上 SAC 和 PPO 对所选超参数的敏感性

表 7: SAC 环境特定参数

Environment	Learning rate schedule
HopperBulletEnv-v0	linear
Walker2dBulletEnv-v0	linear

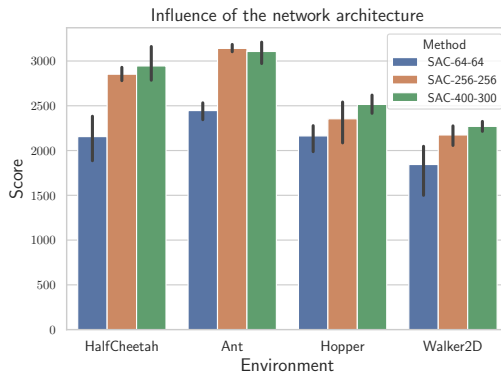


(a) Effect of parallel sampling for PPO

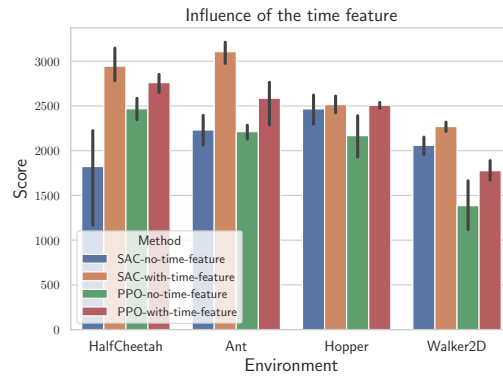


(b) Effect of parallel sampling for A2C

图 12: 噪声矩阵的并行采样对 PyBullet 任务上的 PPO (a) 和 A2C (b) 有积极影响。



(a) Influence of the network architecture



(b) Influence of the time feature

图 13: (a) PyBullet 环境中 SAC 的网络架构 (演员和评论家相同) 的影响。标签显示每层的单元数。(b) PPO 和 SAC 的观测中是否包含时间的影响。

表 8: TD3 超参数

参数值	
<i>Shared</i>	
optimizer	Adam [54]
discount (γ)	0.98
replay buffer size	$2 \cdot 10^5$
number of hidden layers (all networks)	2
number of hidden units per layer	[400, 300]
number of samples per minibatch	100
non-linearity	<i>ReLU</i>
target smoothing coefficient (τ)	0.005
target policy noise	0.2
target noise clip	0.5
policy delay	2
warm-up steps	10 000
normalization	None
<i>gSDE</i>	
initial $\log \sigma$	-3.62
learning rate for TD3	$6 \cdot 10^{-4}$
target update interval	64
train frequency	64
gradient steps	64
learning rate for gSDE	$1.5 \cdot 10^{-3}$
<i>Unstructured Exploration</i>	
learning rate	$1 \cdot 10^{-3}$
action noise type	Gaussian
action noise std	0.1
train frequency	every episode
gradient steps	every episode

表 9: A2C 超参数

参数值	
<i>Shared</i>	
number of workers	4
optimizer	RMSprop with $\epsilon = 1 \cdot 10^{-5}$
discount (γ)	0.99
number of hidden layers (all networks)	2
number of hidden units per layer	[64, 64]
shared network between actor and critic	False
non-linearity	<i>Tanh</i>
value function coefficient	0.4
entropy coefficient	0.0
max gradient norm	0.5
learning rate schedule	linear
normalization	observation and reward [48]
<i>gSDE</i>	
number of steps per rollout	8
initial log σ	-3.62
learning rate	$9 \cdot 10^{-4}$
GAE coefficient [43] (λ)	0.9
orthogonal initialization [49]	no
<i>Unstructured Exploration</i>	
number of steps per rollout	32
initial log σ	0.0
learning rate	$2 \cdot 10^{-3}$
GAE coefficient [43] (λ)	1.0
orthogonal initialization [49]	yes

表 10: PPO 超参数

Parameter	Value
<i>Shared</i>	
optimizer	Adam [54]
discount (γ)	0.99
value function coefficient	0.5
entropy coefficient	0.0
number of hidden layers (all networks)	2
shared network between actor and critic	False
max gradient norm	0.5
learning rate schedule	constant
advantage normalization [48]	True
clip range value function [49]	no
normalization	observation and reward [48]
<i>gSDE</i>	
number of workers	16
number of steps per rollout	512
initial log σ	-2
gSDE sample frequency	4
learning rate	$3 \cdot 10^{-5}$
number of epochs	20
number of samples per minibatch	128
number of hidden units per layer	[256, 256]
non-linearity	<i>ReLU</i>
GAE coefficient [43] (λ)	0.9
clip range	0.4
orthogonal initialization [49]	no
<i>Unstructured Exploration</i>	
number of workers	1
number of steps per rollout	2048
initial log σ	0.0
learning rate	$2 \cdot 10^{-4}$
number of epochs	10
number of samples per minibatch	64
number of hidden units per layer	[64, 64]
non-linearity	<i>Tanh</i>
GAE coefficient [43] (λ)	0.95
clip range	0.2
orthogonal initialization [49]	yes

表 11: PPO 环境特定参数

Environment	Learning rate schedule	Clip range schedule	initial log σ
<i>gSDE</i>			
AntBulletEnv-v0	default	default	-1
HopperBulletEnv-v0	default	linear	-1
Walker2dBulletEnv-v0	default	linear	default
<i>Unstructured Exploration</i>			
Walker2dBulletEnv-v0	linear	default	default